	PCET'S Pimpri Chinchwad College of Engineering, Pune

Name: Soham kulkarni **PRN**: 124B1F056

Course: Design and Analysis of Algorithm Laboratory

ASSIGNMENT NO. 02

---

1. Problem Statement

Movie Recommendation System Optimization A popular OTT platform, StreamFlix, offers personalized recommendations by sorting movies based on user preferences, such as IMDB rating, release year, or watch time popularity. However, during peak hours, sorting large datasets slows down the system. As a backend engineer, you must:

- Implement Quick sort to efficiently sort movies based on various user-selected parameters.
- Handle large datasets containing of movies while maintaining fast response times

2. Course Objective

- To know the basics of computational complexity of various algorithms.
- To select appropriate algorithm design strategies to solve real-world problems.

3. Course Outcome

- Analyze the asymptotic performance of algorithms.
- Solve computational problems by applying suitable paradigms of Divide and Conquer or Greedy methodologies

4. Theory

Quick Sort is one of the most widely used sorting algorithms because of its efficiency, simplicity in concept, and adaptability. It is based on the divide-and-conquer paradigm, which makes it highly effective in sorting large datasets. Unlike some other sorting algorithms, Quick Sort is an in-place algorithm, meaning it requires very little extra memory, and it avoids the overhead of merging, as seen in Merge Sort.

4.1. The working of Quick Sort can be explained in three major steps

1. Divide: A pivot element is selected from the array. The choice of pivot can significantly affect performance. Common strategies include choosing the last element, the first element, a random element, or using the median-of-three method (the median of the first, middle, and last elements).

2. Partition: The array is rearranged so that all elements smaller than the pivot are placed to its left and all elements larger are placed to its right. The pivot itself is placed in its correct sorted position.

3. Conquer: The same process is applied recursively to the left and right subarrays created by the partition step. Unlike some other divide-and-conquer algorithms, Quick Sort does not require a separate combine step. Once all recursive calls complete, the array is already sorted due to the placement of pivots at each stage.

The efficiency of Quick Sort lies in its average-case time complexity of O(n log n), which makes it suitable for a wide range of applications. However, in the worst case—when the pivot selection is poor and the partitioning is unbalanced—its performance can degrade to O(n2). To minimize the likelihood of this scenario, several optimizations are often applied:

4. Random Pivot Selection: Choosing a random pivot reduces the chance of encountering consistently unbalanced partitions.

- Median-of-Three Method: Using the median of the first, middle, and last elements as the

pivot leads to more balanced partitions in practice.

- Tail Recursion Elimination: By converting the tail-recursive calls into iterative steps, the recursion depth can be reduced, improving space efficiency.
- Hybrid Approach: For small subarrays, switching from QuickSort to a simpler algorithm like Insertion Sort can improve performance since the overhead of recursion becomes significant for small input sizes.

4.2. Quick Sort Algorithm

i. Start with an unsorted array.

ii. Choose a pivot element (last element, first element, random, or median-of-three).

iii. Partition the array so that:

iv. All elements smaller than the pivot are placed before it.

v. All elements greater than the pivot are placed after it.

vi. Place the pivot in its correct sorted position.

vii. Recursively apply QuickSort on the left subarray and right subarray.

viii. Continue until subarrays have size 0 or 1 (base case).

ix. End when the entire array is sorted.

4.3. Pseudo Code for Quick Sort

```
Procedure QUICK_SORT(arr, low, high):
    if low < high then
        pivotIndex ← PARTITION(arr, low, high)
        // Recursively sort elements before and after partition
        QUICK_SORT(arr, low, pivotIndex - 1)
        QUICK_SORT(arr, pivotIndex + 1, high)
```

```
        end if
End Procedure

Procedure PARTITION(arr, low, high):
        pivot ← arr[high] // choose last element as
        pivot i ← low - 1 // index of smaller element
        for j ← low to high - 1 do
                if arr[j] ≤ pivot then
                        i ← i + 1
                        SWAP(arr[i], arr[j])
                end if
        end for

        SWAP(arr[i + 1], arr[high]) // place pivot at correct
        position return (i + 1) // pivot index
End Procedure
```

4.4 Example to clearly see how Quick Sort works

Example Array: [3, 1, 4, 2]

Step 1: Choose pivot

We choose the last element →pivot = 2.

Step 2: Partition around pivot Compare each element with 2:

3 > 2 → goes to right

1 < 2 → goes to left

4 > 2 ⟶ goes to right

After partition: [1, 2, 4, 3]

Pivot 2 is in its correct sorted position.

Step 3: Recursive calls

Left subarray: [1] ⟶ already sorted

Right subarray: [4, 3]

Step 4: Quick Sort on [4, 3]

Pivot = 3

Compare:

4 > 3 ⟶ goes right

After partition: [3, 4]

Final Sorted Array

Combine results: [1, 2, 3, 4]

5. OUTPUT

Github Link:https://github.com/SOHAM0007-CODER/DAA_ASSIGNMNET

6. Conclusion

Quick Sort is an efficient and memory-friendly algorithm ideal for sorting large movie datasets based on user preferences like rating, release year, or popularity. By using Quick Sort, StreamFlix can ensure fast and responsive recommendations even during peak hours, improving performance and user experience.